

Alur Autentikasi JWT

Dari `go run ./cmd/cli admin create` sampai token itu dibaca ulang oleh server — apa itu JWT, dari mana rahasianya, dan kenapa satu request bisa ditolak dengan “token tidak ditemukan”.

REST API · dasar

JWT · HS256

go-api-starter

A Sebelum masuk ke token: apa itu API

Kalau selama ini kamu pakai aplikasi lewat tampilan (browser, aplikasi desktop), API adalah cara yang sama tapi tanpa tampilan — cuma teks bolak-balik lewat HTTP.

SATU REQUEST, SATU RESPONSE

Setiap kali kamu `curl` atau klik “Send” di Postman, yang terjadi selalu dua bagian: kamu kirim **request** (method + alamat + kadang data), server balas **response** (status + data). Contoh nyata dari sesi kamu tadi:

REQUEST

```
POST /api/v1/admin/auth/login
Content-Type: application/json

{
  "email": "superadmin@mail.com",
  "password": "Khc_jFrEhpw1x-syH1id"
}
```

RESPONSE · 200 OK

```
{
  "success": true,
  "data": {
    "access_token": "eyJhbG...",
    "refresh_token": "wo9Lo...",
    "expires_in": 900
  }
}
```

Tidak ada yang tersimpan di server tentang “kamu sedang login” — HTTP itu *stateless*, server lupa kamu begitu response dikirim. Supaya request berikutnya tetap dikenali sebagai “yang tadi login”, server memberimu sebuah **token** di response ini, dan kamu wajib

menyertakannya lagi di setiap request berikutnya. Token itulah JWT.

1 Anatomi token yang kamu dapat

JWT (*JSON Web Token*) bukan data acak — dia tiga bagian yang digabung tanda titik, dan dua bagian pertama bisa dibaca siapa saja tanpa kunci apa pun. Ini persis token dari login kamu tadi, dipotong per bagian:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlM2NiYTtyZi00ODIwLTQ5NTYtOWQ2Mi00MGVlM2QzZjRhYjYiLCJleHAiOjE3ODY4MTMxMDksImhhdCI6MTc4NjgxMjIwSwic2NvcGUiOiJwbGF0Zm9ybSJJ9.Va7o8arwIAX8245yCnxWAHMAKagizrhTfSa2lJe4eIc
```

HEADER

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD (CLAIMS)

```
{
  "sub": "e3cba62f- ... ",
  "scope": "platform",
  "iat": 1786812209,
  "exp": 1786813109
}
```

SIGNATURE

```
HMACSHA256(
  header + "." + payload,
  JWT_SECRET
)
```

Perhatikan: **header** dan **payload** cuma base64 — bukan dienkripsi, cuma dikodekan. Siapa pun yang mencegat token itu bisa membacanya (coba tempel bagian tengahnya ke decoder JWT manapun, isinya persis tabel di atas). Yang *tidak bisa* dipalsukan orang lain adalah bagian ketiga: **signature**. Itu satu-satunya bagian yang butuh `JWT_SECRET` untuk dihasilkan — server menandatangani header+payload pakai secret itu, dan nanti memverifikasi ulang tanda tangannya di setiap request. Kalau satu karakter saja di payload diubah, signature tidak akan cocok lagi.

2 Dari mana `JWT_SECRET` berasal

Tidak ada yang “didapat dari mana-mana” secara ajaib — kamu sendiri yang menaruhnya, sewaktu setup awal di `docs/getting-started.md` langkah 3:

```
# .env.example → disalin jadi .env
JWT_SECRET=change-this-to-a-random-64-char-string
ACCESS_TOKEN_TTL=15m
```

REFRESH_TOKEN_TTL=168h

Dari file `.env` itu, begini rantainya sampai dipakai:

```
config.Load() · internal/config/config.go
```

Baca `.env` lewat `godotenv.Load()`, ambil `os.Getenv("JWT_SECRET")`. Kalau kosong atau kurang dari 32 karakter, aplikasi **menolak jalan** — bukan diam-diam pakai default, supaya deploy yang lupa ganti secret dari `.env.example` ketahuan sejak awal, bukan pas sudah kena serangan.

```
auth.NewTokenManager(secret, ttl) · cmd/api/main.go
```

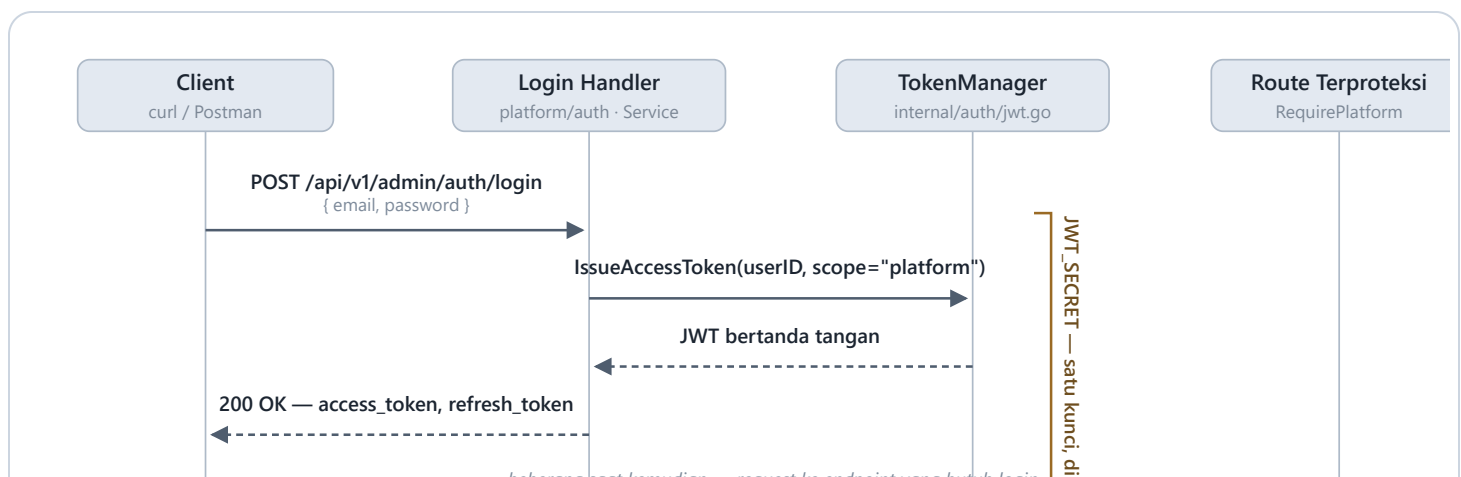
Nilai string itu dibungkus jadi `[]byte` dan disimpan di dalam satu struct `TokenManager` yang hidup selama proses `api.exe` berjalan — dibuat sekali saat startup, dipakai berulang-ulang untuk setiap request, tidak pernah dibaca ulang dari file.

```
TokenManager.IssueAccessToken() & .Verify() · internal/auth/jwt.go
```

Secret yang sama dipakai untuk dua arah: **menandatangani** token baru saat login, dan **memverifikasi** tanda tangan token yang datang di request berikutnya. Ini kunci simetris (HMAC) — bukan sepasang kunci publik/privat — jadi secret ini betul-betul rahasia server, tidak boleh pernah dikirim ke client dalam bentuk apa pun.

3 Satu secret, dua momen pemakaian

Diagram ini menelusuri dua request yang sudah kamu jalankan sendiri malam ini: login untuk *mendapat* token, lalu request lain untuk *memakai* token itu. Perhatikan `TokenManager` di tengah — dipanggil dua kali, dengan secret yang sama, untuk dua tujuan berlawanan.





Dua momen pakai `JWT_SECRET` yang sama: menandatangani token baru saat login (kiri-tengah), lalu memverifikasi tanda tangannya di setiap request ke route terproteksi (kanan). Client tidak pernah melihat secret-nya — hanya token hasil tanda tangan.

4 Kenapa ada dua token

Response login memberi dua string, bukan satu, karena umur dan tugasnya sengaja dibuat berbeda:

ASPEK	ACCESS_TOKEN	REFRESH_TOKEN
Umur (TTL)	15 menit (<code>ACCESS_TOKEN_TTL</code>)	7 hari / 168 jam (<code>REFRESH_TOKEN_TTL</code>)
Dipakai untuk	Header <code>Authorization: Bearer</code> di tiap request ke route terproteksi	Ditukar jadi <code>access_token</code> baru lewat <code>/auth/refresh</code> , saat yang lama kedaluwarsa
Bentuknya	JWT (3 bagian, bisa diverifikasi tanpa tanya database)	String acak 32-byte biasa — <i>bukan</i> JWT
Disimpan di server?	Tidak — cukup diverifikasi tanda tangannya saja, stateless	Ya, hash SHA-256-nya disimpan di tabel <code>refresh_tokens</code> , supaya bisa dicabut (logout)

Alasan dipisah: `access_token` dibuat pendek umur karena kalau bocor, jendela penyalahgunaannya kecil (maks 15 menit) dan server tidak perlu cek database untuk memvalidasinya — cukup hitung ulang signature. `refresh_token` umurnya panjang tapi baru

dipakai sesekali, dan karena disimpan di database (dalam bentuk hash), dia **bisa dicabut sewaktu-waktu** — itulah yang dilakukan endpoint `/auth/logout`: menandai `revoked_at` di baris itu, bukan “menghapus” JWT yang mustahil ditarik balik begitu sudah beredar.

5 Praktiknya, langkah demi langkah

Urutan ini persis yang sudah kamu jalani malam ini, plus dua langkah yang belum kamu coba (pakai token, refresh):

- 1 **Buat admin.** CLI membuat baris di tabel `users` (schema `platform`) dengan password di-hash bcrypt, lalu print password asli sekali saja — tidak pernah disimpan dalam bentuk plain text.

```
go run ./cmd/cli admin create --email=superadmin@mail.com --name="Fauzi"
```

- 2 **Login** — tukar email/password dengan sepasang token.

```
curl -X POST http://localhost:8080/api/v1/admin/auth/login `
-H "Content-Type: application/json" `
-d "{\"email\":\"superadmin@mail.com\",\"password\":\"<password>\"}"
```

- 3 **Simpan `access_token`**, lalu sertakan di setiap request ke route yang butuh login — bukan di body, tapi di header.

```
curl http://localhost:8080/api/v1/users `
-H "Authorization: Bearer <access_token>"
```

- 4 **Sudah 15 menit dan token kedaluwarsa?** Tidak perlu login ulang — tukar `refresh_token` yang masih hidup 7 hari.

```
curl -X POST http://localhost:8080/api/v1/admin/auth/refresh `
-H "Content-Type: application/json" `
-d "{\"refresh_token\":\"<refresh_token>\"}"
```

- 5 **Logout** mencabut `refresh_token` di database (tandanya `revoked_at` terisi) — `access_token` yang sudah beredar tetap sah sampai 15 menit-nya habis sendiri, itu trade-off yang sudah disebut di tabel atas.

```
curl -X POST http://localhost:8080/api/v1/admin/auth/logout `
-H "Content-Type: application/json" `
-d "{\"refresh_token\":\"<refresh_token>\"}"
```

6 Balik ke error kamu tadi

“token tidak ditemukan”

Sekarang pesan itu bisa dibaca persis dari diagram di atas: itu bukan gagal login, dan bukan gagal verifikasi tanda tangan. Itu titik paling awal di jalur kanan diagram — sebelum `TokenManager.Verify()` sempat dipanggil sama sekali — middleware `RequirePlatform` mengecek header `Authorization`, dan headernya kosong. Body JSON `{ "email", "password" }` tidak relevan di titik itu, karena route yang kena middleware ini bukan route login.

Kejadian kamu sebelumnya cocok dengan ini: typo di URL membuat request mendarat di route yang salah (yang butuh token), padahal niatnya memanggil `/auth/login` (yang tidak butuh token sama sekali).